

LAPORAN TUGAS PRAKTIKUM 6
DEADLOCK SISTEM OPERASI



Dosen Pengampu :
Triawan Adi Cahyanto M.kom

Disusun Oleh :
Hafabi Arrohim (2410651013)
Kelas A jumat

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
TAHUN 2025

TUGAS 1 ANALISIS DEADLOCK

1. Mengapa program mengalami deadlock?

Program deadlock_simple.c mengalami deadlock karena:

- Thread 1 memegang Lock 1 lalu menunggu Lock 2
- Thread 2 memegang Lock 2 lalu menunggu Lock 1
- Kedua thread saling menunggu untuk mendapatkan resource yang sedang dipegang thread lain
- Tidak ada thread yang melepaskan lock → kondisi wait forever

Sehingga program macet dan tidak pernah selesai.

2. Kondisi deadlock apa saja yang terpenuhi ?

Menurut Coffman, ada 4 syarat deadlock. Semua syarat terpenuhi di program ini:

Kondisi	Penjelasan
Mutual Exclusion	Lock hanya bisa dipegang satu thread dalam satu waktu.
Hold and Wait	Thread 1 memegang Lock 1 sambil menunggu Lock 2. Thread 2 memegang Lock 2 sambil menunggu Lock 1.
No Preemption	Lock tidak bisa direbut paksa oleh thread yang lain.
Circular Wait	Thread 1 menunggu Lock 2 → Thread 2 menunggu Lock 1 → lingkaran tunggu terbentuk.

Kesimpulannya: semua syarat deadlock terpenuhi dan deadlock terjadi.

3. Gambar Resource Allocation Graph (RAG)

T1 ----holds----> L1

T1 <---needs----- L2

T2 ----holds----> L2

T2 <---needs----- L1

TUGAS 2 IMPLEMENTASI PENCEGAHAN

Memodifikasi program *deadlock_simple.c* menggunakan Teknik :

1. Lock Ordering

Semua thread mengambil lock dalam urutan yang sama: Lock 1 Lock 2.
Dengan ini circular wait dihilangkan

Kode yang dimodifikasi :

```
pthread_mutex_lock(&lock1);
```

```
pthread_mutex_lock(&lock2);
```

hasilnya tidak akan terjadi deadlock dan thread berjalan satu persatu

kesimpulan nya :

lock ordering adalah Teknik paling sederhana dan efektif karena menghilangkan circular wait.

2. Try-Lock dengan Retry

Jika thread gagal mendapatkan lock kedua lock pertama dilepas, lalu thread mencoba ulang.

Kode intinya yang di modifikasi :

```
if (pthread_mutex_trylock(&lock2) == 0) {
```

```
} else {
```

```
    pthread_mutex_unlock(&lock1);
```

```
    retry++;
```

```
}
```

Hasil nya tidak ada deadlock dan thread akan retry sampai berhasil mengambil kedua lock.

Kesimpulanya :

Try-lock menghilangkan kondisi hold and wait karena lock dilepas saat gagal mengambil lock lain.

3. Bandingkan Kedua Metode

Metode	Kelebihan	Kekurangan
Lock Ordering	Sederhana, efektif, deterministik	Semua kode harus konsisten memakai urutan yang sama
Try-Lock	Fleksibel, cocok untuk sistem real-time	Banyak retry → performa bisa turun

Kesimpulan:

- Jika desain sistem stabil gunakan Lock Ordering
- Jika situasi dinamis gunakan Try-Lock

TUGAS 3 BANKER'S ALGORITHM

Data yang di berikan :

- 4 proses
- 3 jenis resource
- Available = [5,4,3]

Kode Program yang akan di masukkan :

```
#include <stdio.h>
#include <stdbool.h>

#define P 4
#define R 3

int available[R] = {5, 4, 3};

int maximum[P][R] = {
    {3, 2, 2},
    {6, 1, 3},
    {3, 1, 4},
    {4, 2, 2}
};

int allocation[P][R] = {
    {1, 0, 0},
    {5, 1, 1},
    {2, 1, 1},
    {0, 0, 2}
};

int need[P][R];

void compute_need() {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = maximum[i][j] - allocation[i][j];
}

bool is_safe() {
    int work[R], finish[P] = {0};
    for (int i = 0; i < R; i++) work[i] = available[i];

    for (int k = 0; k < P; k++) {
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool ok = true;
                for (int j = 0; j < R; j++)
                    if (need[i][j] > work[j]) ok = false;

                if (ok) {
                    for (int j = 0; j < R; j++)
                        work[j] += allocation[i][j];
                    finish[i] = 1;
                }
            }
        }
    }

    for (int i = 0; i < P; i++)
        if (!finish[i]) return false;

    return true;
}

int main() {
    compute_need();
}
```

```
int main() {
    compute_need();

    if (is_safe())
        printf("Sistem dalam kondisi AMAN.\n");
    else
        printf("Sistem TIDAK aman.\n");

    return 0;
}
```

Dan outputnya akan seperti ini :

```
PS1=faby@fabyyar-biee:~/PraktikumOS6/Tugas$ ./Tgs_bankers_algorithm
Sistem dalam kondisi AMAN.
```

Analisis nya :

- Program menghitung need, allocation, dan available
- Sistem kemudian dicek apakah masih ada safe sequence
- Jika semua proses dapat berjalan sistem aman
- Jika salah satu proses tidak dapat dilayani sistem unsafe

Kesimpulannya :

Dengan resource available [5,4,3], sistem berada dalam safe state, sehingga deadlock dapat dihindari.

TUGAS 4 KASUS NYATA (Real Case)

Buat simulasi deadlock untuk kasus :

- Orang A ingin transfer dari Rekening 1 → Rekening 2
- Orang B ingin transfer dari Rekening 2 → Rekening 1
- Jika keduanya lock rekening secara berlawanan → deadlock
- Dan implementasikan deadlock prevention

Kode untuk membuat program deadlock versi tranfers :

```
GNU nano 7.2
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rek2 = PTHREAD_MUTEX_INITIALIZER;

void* transfer_A() {
    printf("A: Mengunci Rekening 1\n");
    pthread_mutex_lock(&rek1);

    sleep(1);

    printf("A: Mencoba mengunci Rekening 2\n");
    pthread_mutex_lock(&rek2);

    printf("A: Transfer berhasil (1 → 2)\n");

    pthread_mutex_unlock(&rek2);
    pthread_mutex_unlock(&rek1);
}

void* transfer_B() {
    printf("B: Mengunci Rekening 2\n");
    pthread_mutex_lock(&rek2);

    sleep(1);

    printf("B: Mencoba mengunci Rekening 1\n");
    pthread_mutex_lock(&rek1);

    printf("B: Transfer berhasil (2 → 1)\n");

    pthread_mutex_unlock(&rek1);
    pthread_mutex_unlock(&rek2);
}

int main() {
    pthread_t A, B;

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    return 0;
}
```

Dan outputnya akan seperti ini :

```
PS1=faby@fabyar-biee:~/PraktikumOS6/Tugas$ ./deadlock_tranfers
A: Mengunci Rekening 1
B: Mengunci Rekening 2
B: Mencoba mengunci Rekening 1
A: Mencoba mengunci Rekening 2
^C
```

Disini menunjukkan jika program yang saya buat itu HANG/ Macet.

Kemudia ini kode untuk program pencegahan deadlock :

```
GNU nano 7.2
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rek2 = PTHREAD_MUTEX_INITIALIZER;

void lock_ordered() {
    pthread_mutex_lock(&rek1);
    pthread_mutex_lock(&rek2);
}

void unlock_ordered() {
    pthread_mutex_unlock(&rek2);
    pthread_mutex_unlock(&rek1);
}

void* transfer_A() {
    printf("A: Mengunci rekening berurutan (1 → 2)\n");
    lock_ordered();
    printf("A: Transfer aman (1 → 2)\n");
    unlock_ordered();
}

void* transfer_B() {
    printf("B: Mengunci rekening berurutan (1 → 2)\n");
    lock_ordered();
    printf("B: Transfer aman (2 → 1)\n");
    unlock_ordered();
}

int main() {
    pthread_t A, B;

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    return 0;
}
```

Dan output yang di hasilkan akan seperti ini :

```
PS1=faby@fabyyar-biee:~/PraktikumOS6/Tugas$ ./prevention_tranfers
A: Mengunci rekening berurutan (1 → 2)
A: Transfer aman (1 → 2)
B: Mengunci rekening berurutan (1 → 2)
B: Transfer aman (2 → 1)
```

Nah disini tampak jelas bahwa itu tidak ada deadlock dan tranfers aman.

Kesimpulannya :

Kasus nyata seperti transfer bank sangat mungkin mengalami deadlock. Dengan teknik lock ordering, deadlock dapat dicegah dan transaksi menjadi aman.